

Take-Home Case: Technical Lead, Data Science

Business Case: Optimizing Multi-Bank Liquidity and Cash Flow

Context

A fintech company holds balances across **multiple banks and currencies** to serve its clients' payment needs. Money is spread across different accounts, and needs to move between them proactively — before a shortfall blocks a payment or a surplus sits idle earning nothing. Today, rebalancing decisions are made reactively, and leadership wants a data-driven way to decide **when, how much, and where** to move money.

You've just joined the team as Technical Lead, Data Science. A squad attempted a first version of a liquidity forecasting/rebalancing model, but it's stuck — the forecasts are unreliable, and the recommended transfers don't always make sense. You're asked to step in directly: not just advise, but get hands-on this week and propose a clearer path forward.

You're given:

1. **accounts.csv** — a catalog of 6 accounts (COP, USD, MXN) across different banks.
2. **account_balances_daily_RAW.csv** — daily balances, inflows, and outflows per account over ~9 months.
3. **transfers_log_RAW.csv** — historical transfers between accounts, with the request date, settlement date, and fee for each transfer.
4. **flawed_model_squad_draft.ipynb** — the squad's first-attempt notebook: a balance forecast and a rebalancing rule. The notebook runs without error and concludes the approach "looks like it's working" — this is your starting point for the analysis.

Your task

Part 1 — Diagnose and fix (hands-on). Review the notebook and the data provided. Identify what's wrong with the squad's forecasting/rebalancing approach and fix it yourself: don't just point out the problem, rebuild whatever part needs it. Explain in plain terms what was wrong and why it matters for a real cash-management decision (e.g., risking a shortfall vs. moving money unnecessarily).

Part 2 — Design a better approach to the actual question. Propose (and implement, at least in simplified form) a sound method to answer: *given forecasted balances and flows across accounts/currencies, when and how much should be moved to minimize the risk of shortfalls while minimizing unnecessary transfers and their costs?* This can draw on forecasting (time

series), optimization (e.g., a simple constrained allocation/rebalancing rule), or a hybrid — the point is that the method should be explainable and its assumptions stated clearly.

Part 3 — Explain it to two audiences. Write two short explanations of your approach and recommendation:

- One for the squad's data scientist (technical: methodology, assumptions, and where the model could still fail).
- One for a Treasury/Finance stakeholder (non-technical: what this means for cash safety and cost, and what residual risk remains).

Part 4 — Show your ways of working. Briefly note where (if at all) you used AI tools (coding assistants, LLM agents) during this exercise, and where you deliberately didn't.

Part 5 — How would you productionize this? We're not asking you to deploy it — just to describe, in half a page, how you'd take this from a notebook to something that runs reliably in production. At minimum, cover:

- **Data pipeline:** where would clean data come from in production, and what quality checks would you run before feeding the model?
- **Cadence and orchestration:** how often would the forecast/rebalancing run, and what tool would you use to orchestrate it?
- **Monitoring:** what metrics would you watch to know if the model is degrading?
- **Safeguards:** what controls would prevent a bad recommendation from executing automatically?

Submission Guidelines

1. **Corrected analysis** (Python notebook or script).
2. **Short write-up** (1–2 pages or notebook markdown) with the explanations from Part 3, the note from Part 4, and the proposal from Part 5.
3. **Submission format:** Jupyter/Colab notebook or PDF report. A GitHub repo is a plus.
4. **Turnaround time:** 5 business days.

Test Files

- `accounts.csv`
- `account_balances_daily_RAW.csv`
- `transfers_log_RAW.csv`
- `flawed_model_squad_draft.ipynb`

All of these are 100% synthetic.